```
    for point in node.points
do                                                    #3
      dist ← distance(point, target)
                                          #4
      if dist < nnDist
then                                                      #5
        (nnDist, nn) ← (dist,
point)                                               #6

else

    sortedChildren ← sortNodesByDistance(node.children,
target)               #7
    for child in sortedChildren
do                                                  #8
      if nodeDistance(child, target) < nnDist
then                              #9
        (nnDist, nn) ← nearestNeighbor(child, target, (nnDist,
nn))           #10
  return (nnDist,
nn)                                                                #11
```

#1 Function `nearestNeighbor` returns the closest point to a given target. It takes the node to search and the query point. We also (optionally) pass the best values found so far for nearest neighbor (*NN*) and its distance, to help pruning. These values default to `null`, `infinity` for a call on the tree root, unless we want to limit the search inside a spherical region (in that case, just pass the sphere's radius as the initial value for `nnDist`)

#2 Check if node is a leaf. Leaves are the only nodes containing points, and so the only nodes where we possibly update the nearest neighbor found.

#3 Cycle through all points in this leaf.

#4 Compute the distance between current point and `target`.